

CSA0208-C Programming

Automobile Service Centre Management System

SUBMITTED BY:

ISKA VARSHINI(192320011)

S PAVITHRASHA(192320016)

T LAKSHMI PRASANNA(192525207)

1. Problem Statement

The automobile industry is a major service-oriented sector where regular vehicle maintenance and servicing are essential for ensuring vehicle safety and customer satisfaction. Automobile service centres manage a large amount of information, including customer details, vehicle numbers, vehicle models, service types, service dates, replacement parts, labour charges, and total billing information. In many small and medium-sized service centres, these records are still maintained manually using notebooks, registers, and paper documents.

Maintaining records manually creates several difficulties. Searching for a specific vehicle record becomes time-consuming as the number of customers increases. Updating records manually can result in incorrect or overwritten information. Calculating parts costs, labour charges, and total bills manually may also cause billing errors. Physical records can be lost, damaged, or become difficult to manage over time. Preparing daily service reports or identifying vehicles with high service costs also requires additional manual effort.

To overcome these problems, an **Automobile Service Centre Management System** is proposed using the **C programming language**. The system provides a simple menu-driven interface for managing vehicle service records digitally. It stores important details such as vehicle number, customer name, vehicle model, service type, service date, parts cost, labour cost, and total service cost.

The system uses **structures** to organize related vehicle and service information into individual records. **Pointers to structures** are used for performing operations such as updating records and calculating service costs efficiently. The application allows users to register vehicles, search for records using vehicle numbers, update service details, calculate total bills, sort records based on service cost, display vehicles exceeding a specified service-cost limit, and generate daily service summaries.

File handling is also implemented to provide permanent storage of service records. Records can be saved to a file and automatically loaded when the application is started again. This eliminates the need to enter previously stored information repeatedly.

The system also includes various **validation and error-handling mechanisms** to improve reliability. Duplicate vehicle numbers are identified to prevent repeated records. Negative values for parts and labour costs are rejected as invalid inputs. Service dates are validated to ensure that months, days, and calendar values are correct. The system also checks the maximum record capacity to prevent array overflow. File operations are verified to ensure that files are successfully opened before reading or writing data.

Thus, the **Automobile Service Centre Management System** demonstrates the practical application of important C programming concepts such as **structures, pointers, arrays, functions, searching, sorting, file handling, data validation, and error handling**. By replacing manual record maintenance with a computerized approach, the system improves the speed, accuracy, reliability, and overall efficiency of automobile service record management.

2. Objective

1. Electronic Service Record Management:

To store customer, vehicle, service, date, and billing information electronically instead of using manual registers.

2. Structured Data Organization:

To use structures and nested structures in C to organize vehicle, owner, service, and billing information efficiently.

3. Pointer-Based Record Manipulation:

To apply pointers to access and modify vehicle service records efficiently without unnecessary data duplication.

4. Automated Service Cost Calculation:

To calculate the total service cost automatically based on selected services, spare parts, labor charges, and other applicable costs.

5. Vehicle Service Registration:

To provide a registration facility for entering and storing complete vehicle and service-related details.

6. Vehicle Record Searching:

To implement an efficient search mechanism for retrieving vehicle service records using the vehicle registration number.

7. Service Record Modification:

To allow existing service details to be updated using pointers to structures.

11. Permanent Data Storage:

To use file handling techniques to permanently store vehicle service records and prevent data loss when the program is closed.

12. Automatic Record Retrieval:

To load previously stored service records automatically when the program is restarted.

13.Input Validation and Error Management: To validate user inputs and handle invalid entries, incorrect dates, invalid costs, and unavailable vehicle records appropriately.

14. Menu-Driven User Interface :

To provide a simple and user-friendly menu-driven interface for performing registration, searching, updating, sorting, reporting, and other operations.

Overall Objective

To develop a reliable and efficient **Automobile Service Centre Management System in C** that integrates structures, nested structures, arrays, pointers, functions, searching, sorting, file handling, input validation, and error handling to manage vehicle service records and billing effectively.

3. Requirements and Environment Used

The **Automobile Service Centre Management System** is developed using the **C programming language**. The project requires a standard computer, a C compiler, and a suitable development environment for writing, compiling, executing, testing, and debugging the application.

Hardware Requirements

The minimum hardware requirements for running and developing the application are:

- Computer or laptop
- Minimum 4 GB RAM
- At least 1 GB of free storage
- Standard keyboard and display
- Any modern processor capable of running a C compiler

Since the application is a console-based C program, it requires only basic processing power and memory and does not need high-end hardware.

Software Requirements

The software requirements include:

- **Programming Language:** C
- **Compiler:** GCC or MinGW GCC
- **Operating System:** Windows, Linux, or macOS
- **Development Environment:** Visual Studio Code, Code::Blocks, Dev-C++, or any compatible C IDE
- **Header Files:** stdio.h, string.h, and ctype.h

Programming Environment

The program can be compiled using GCC with the following command:

```
gcc automobile_service.c -o automobile_service
```

After successful compilation, the application can be executed using:

```
automobile_service
```

The system is designed as a console application, so it does not require external database software or GUI frameworks.

Data Storage Requirement

The application uses a binary file named **service_records.dat** to store service records permanently. Binary file storage allows the complete structure to be efficiently written and retrieved using the standard C functions `fwrite()` and `fread()`.

Maximum Record Capacity

The system supports a maximum of **100 service records**. This limit is suitable for an academic project and allows the application to handle a moderate amount of data without requiring dynamic memory allocation.

The maximum capacity can be increased or modified in the future according to the requirements of the service centre.

4. Design / Proposed Solution

The proposed Automobile Service Centre Management System follows a **modular, menu-driven approach**. The application is divided into separate functions, with each function responsible for a specific operation. This improves readability, simplifies debugging, supports easier maintenance, and allows different modules to be developed and tested independently.

4.1 Data Representation

Each vehicle service is represented using a structure called **ServiceRecord**. It stores information related to the customer, vehicle, service details, and service charges.

The structure contains:

- Vehicle Number
- Customer Name
- Vehicle Model
- Service Type
- Service Date
- Parts Cost
- Labour Cost
- Total Service Cost

A separate **Date** structure stores the day, month, and year. This makes date validation and date-based operations easier.

The data can be conceptually organized as:

ServiceRecord

- Vehicle Number
- Customer Name
- Vehicle Model
- Service Type
- Service Date
 - Day
 - Month
 - Year
- Parts Cost
- Labour Cost
- Total Service Cost

4.2 Record Storage

During program execution, all service records are stored in an array of ServiceRecord structures.

The maximum capacity is defined as:

MAX_RECORDS = 100

A variable named count keeps track of the number of records currently stored.

The records are maintained as:

```
records[0]
records[1]
records[2]
...
records[count - 1]
```

Before adding a new record, the system checks whether the maximum capacity has been reached.

4.3 Vehicle Registration Module

The registration module is responsible for adding new vehicle service records.

The module performs the following steps:

1. Checks the available record capacity.
2. Accepts the vehicle number.
3. Converts the vehicle number to uppercase.

4. Checks whether the vehicle number already exists.
5. Accepts customer details.
6. Accepts vehicle and service information.
7. Validates the service date.
8. Accepts and validates the parts cost.
9. Accepts and validates the labour cost.
10. Calculates the total service cost.
11. Stores the completed record in the array.

This process prevents duplicate and invalid information from being stored.

4.4 Searching Module

The searching module uses the **vehicle number** as the unique identifier.

A linear search is performed through the available records. The entered vehicle number is compared with each stored vehicle number using the strcmp() function.

If a matching record is found, its position is returned. If no match is found, the function returns -1.

4.5 Pointer-Based Calculation Module

The total service cost is calculated using a **pointer to a structure**.

The calculation is:

Total Service Cost = Parts Cost + Labour Cost

The address of the ServiceRecord structure is passed to the calculation function. The pointer accesses the structure members directly using the arrow (->) operator.

This avoids unnecessary copying of the complete structure and allows the calculated value to be stored directly in the original record.

4.6 Updating Module

The update module first searches for the required vehicle using its vehicle number.

When the record is found, its address is passed to the update function. The function then modifies the required information directly through a structure pointer.

If the parts cost or labour cost is changed, the total service cost is recalculated automatically.

4.7 Sorting Module

The system uses **Insertion Sort** to arrange service records according to their total service cost.

The records are sorted in **descending order**, so the vehicle with the highest service cost appears first.

This makes it easier to identify high-value service transactions.

4.8 High-Cost Analysis Module

The user can enter a specific service-cost threshold.

The system compares the total service cost of every record with the given amount.

If:

Total Service Cost > Specified Amount

the corresponding vehicle information is displayed.

This feature helps identify vehicles that require expensive repairs or services.

4.9 Daily Summary Module

The user can enter a particular service date. The system checks all stored records and identifies the records matching that date.

For the selected date, the system calculates:

- Number of vehicles serviced
- Total service revenue

The results are displayed as a daily service summary.

4.10 File Handling Module

File handling is used to provide permanent storage of service records.

When the application starts:

Load Records → File

When the user saves the records or exits the application:

Save Records → File

Before performing any file operation, the program checks whether the file was opened successfully.

The record count read from the file is also validated to prevent invalid memory or array access caused by corrupted data.

4.11 Error Handling Module

The system handles several possible errors, including:

- Duplicate vehicle numbers
- Negative service costs
- Invalid dates
- Invalid menu choices
- Maximum record capacity
- Missing data files
- Invalid or corrupted record counts

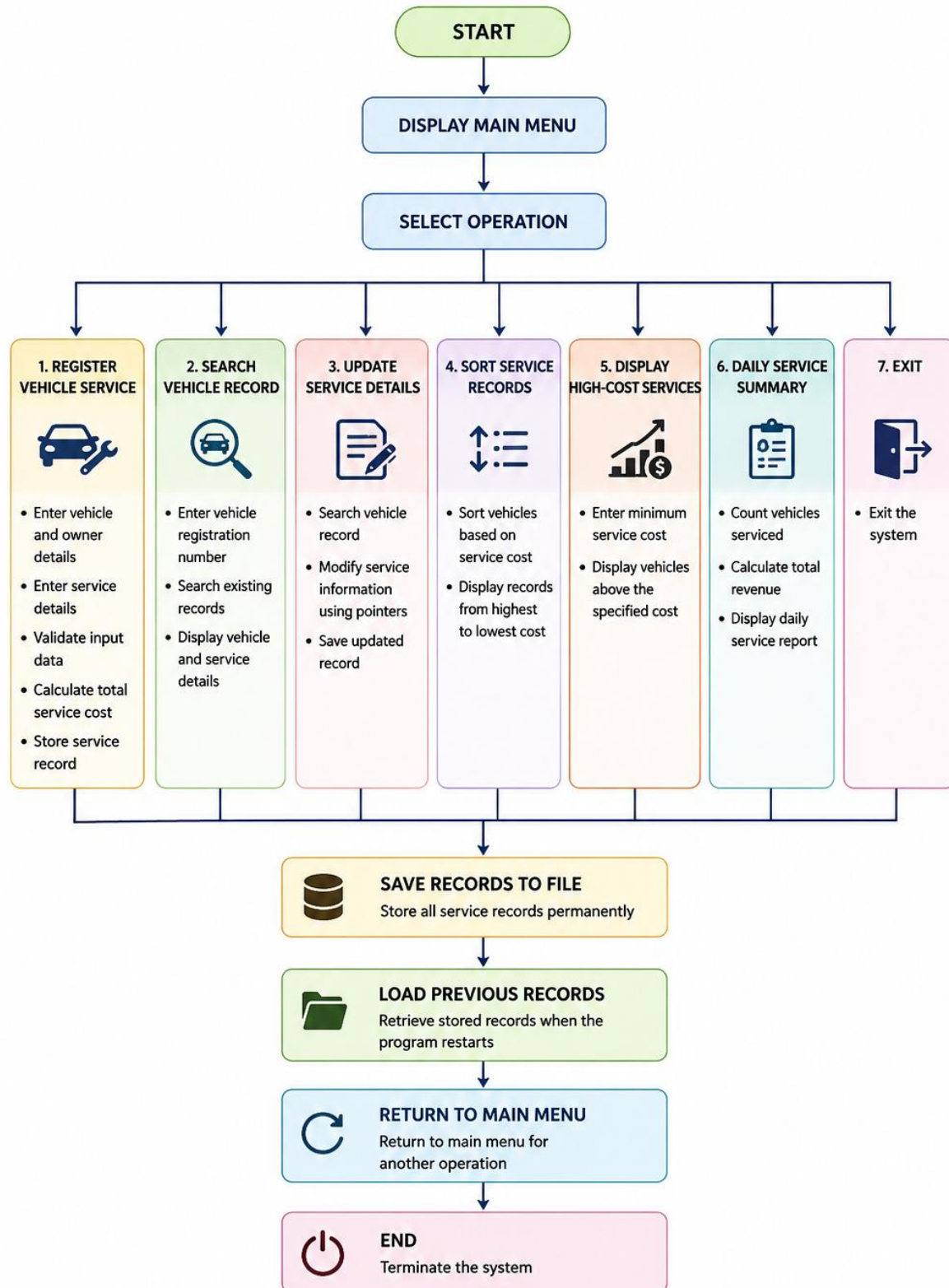
These validation mechanisms improve the reliability and safety of the application.

Overall System Flow

The overall operation of the system can be summarized as:

OVERALL SYSTEM FLOW

AUTOMOBILE SERVICE CENTRE MANAGEMENT SYSTEM



5. Algorithm / Pseudocode

5.1 Main Program Algorithm

1. Begin execution of the program.
2. Create an array to store multiple ServiceRecord structures.
3. Set the initial number of records to zero.
4. Read and retrieve existing service records from the data file.
5. Present the available operations through the main menu.
6. Read the operation selected by the user.
7. Perform the corresponding operation based on the selected menu option.
8. After completing the operation, return to the main menu.
9. When the Exit option is selected, write the current records into the file.
10. Close the program successfully.

5.2 Vehicle Service Registration Algorithm

1. Verify whether space is available for a new service record.
2. If the record limit is reached, display a capacity warning and stop the registration process.
3. Read the vehicle registration number from the user.
4. Convert the entered registration number into uppercase for consistent comparison.
5. Check the existing records to identify duplicate vehicle numbers.
6. If the vehicle number already exists, display a duplicate-entry message and terminate registration.
7. Read the customer's name.
8. Enter the vehicle model information.
9. Select or enter the required service type.
10. Enter the date on which the service is performed.
11. Check whether the entered date is valid.
12. If the date is incorrect, prompt the user to enter a valid date.
13. Enter the amount spent on spare parts.
14. Check the parts cost and reject negative values.
15. Enter the labour charges.
16. Validate the labour cost and reject negative values.
17. Compute the total service amount using the parts and labour costs.
18. Add the completed service record to the array.
19. Increment the total number of stored records.
20. Display a confirmation indicating that the vehicle has been registered successfully.

5.3 Vehicle Search Algorithm

1. Request the vehicle registration number from the user.
2. Convert the entered number into uppercase.
3. Begin checking the records from the first array element.
4. Compare the entered registration number with each stored vehicle number.
5. When a matching record is identified, return its array index.
6. Continue the comparison until all available records have been examined.
7. If no matching vehicle is identified, return -1 to indicate that the record does not exist.

5.4 Service Bill Calculation Algorithm

1. Pass the address of the required service record to the calculation function.
2. Use the pointer to retrieve the parts cost.
3. Use the same pointer to retrieve the labour cost.
4. Add the parts cost and labour cost.
5. Store the calculated amount in the total service cost field.
6. Return the calculated result to the calling function.

Formula:

Total Service Cost = Parts Cost + Labour Cost

5.5 Service Record Update Algorithm

1. Obtain the vehicle registration number that needs to be modified.
2. Search the records using the entered vehicle number.
3. If the vehicle record is unavailable, display an appropriate error message.
4. If the record exists, obtain its memory address.
5. Pass the address of the selected record to the update function.
6. Enter the revised customer, vehicle, and service information.
7. Perform validation on the updated date and cost values.
8. Recalculate the total service cost using the updated values.
9. Store the modified information and display a successful update message.

5.6 Service Cost Sorting Algorithm

The system applies Insertion Sort to organize service records according to their total service cost.

1. Begin processing from the second service record.
2. Select the current record and store it as the key element.
3. Compare the key with the records positioned before it.
4. If a previous record has a smaller service cost, shift that record one position forward.
5. Continue comparing until the appropriate position for the key is identified.
6. Insert the key record into its correct position.
7. Repeat the process for every remaining record.
8. Display the sorted records after the complete sorting operation.

5.7 Daily Service Summary Algorithm

1. Request the service date for which the summary is required.
2. Verify that the entered date is valid.
3. Initialize the serviced vehicle count to zero.
4. Initialize the total revenue amount to zero.
5. Examine each service record stored in the system.
6. Compare the service date of each record with the selected date.
7. If the dates are the same, increase the vehicle count by one.
8. Add the corresponding service cost to the total revenue.
9. Continue the process until all service records have been checked.

10. Display the total number of vehicles serviced on the selected date.
11. Display the total revenue generated for that date.

6. Implementation / Source Code Explanation

The Automobile Service Centre Management System is implemented in **C** as a menu-driven console application. A modular programming approach is followed, where different operations are implemented as separate functions. This improves code readability, maintainability, testing, and debugging.

The major C programming concepts used in the implementation include:

- Structures
- Nested structures
- Arrays of structures
- Pointers to structures
- Functions
- String handling
- Searching
- Sorting
- File handling
- Input validation
- Runtime error handling

During execution, service records are maintained in memory and can be permanently stored in a binary file. When the application starts, previously saved records are loaded automatically, allowing data to remain available across multiple executions.

6.1 Header Files Used

The implementation uses the following header files:

- `stdio.h`
- `string.h`
- `ctype.h`

stdio.h

The `stdio.h` header provides standard input/output and file-handling functions such as:

- `printf()`
- `scanf()`
- `fgets()`
- `fopen()`
- `fread()`
- `fwrite()`
- `fclose()`

string.h

The string.h header provides string manipulation functions such as:

- strcmp()
- strcspn()

These functions are used for comparing vehicle numbers and removing newline characters obtained through fgets().

ctype.h

The ctype.h header provides character-processing functions.

The toupper() function is used to convert vehicle numbers into uppercase. This ensures that differences in letter case do not result in duplicate records.

For example:

```
tn01ab1234
TN01AB1234
Tn01Ab1234
```

are all converted to:

```
TN01AB1234
```

This improves the accuracy of searching and duplicate detection.

6.2 Constants Used

The program defines constants for the maximum record capacity and file name:

```
#define MAX_RECORDS 100
#define FILE_NAME "service_records.dat"
```

MAX_RECORDS specifies the maximum number of service records that can be stored in the array.

FILE_NAME provides a fixed name for the data file, making it easier to modify the file name in the future.

6.3 Date Structure

A separate structure is used to store service dates:

```
typedef struct {
    int day;
    int month;
    int year;
} Date;
```

Using a separate Date structure keeps the program organized. Instead of storing a complete date as one string, the day, month, and year are stored separately.

This provides several benefits:

- Easier date comparison
- Easier date validation
- Simplified daily-summary generation
- Support for leap-year validation
- Clearer data organization

6.4 Main ServiceRecord Structure

The primary structure used by the system is ServiceRecord.

It contains:

- vehicleNumber
- customerName
- vehicleModel
- serviceType
- serviceDate
- partsCost
- labourCost
- totalServiceCost

The structure stores all important information related to a vehicle service.

Vehicle Number

The vehicle number acts as the unique identifier of the service record.

Example:

TN01AB1234

The system prevents duplicate vehicle numbers.

Customer Name

Stores the name of the vehicle owner or customer.

Vehicle Model

Stores the model of the vehicle.

Examples:

- Hyundai i20
- Honda City

- Tata Nexon

Service Type

Stores the type of service performed.

Examples include:

- General Service
- Engine Repair
- Oil Change
- Brake Service
- Full Service

Service Date

The nested Date structure stores the day, month, and year of the service.

Parts Cost

Stores the total cost of parts used during servicing.

Labour Cost

Stores the charges for labour or technician services.

Total Service Cost

The total amount is calculated automatically using:

Total Service Cost = Parts Cost + Labour Cost

6.5 Function-Based Program Design

The program is divided into several functions, including:

- convertToUpper()
- clearInputBuffer()
- isValidDate()
- isDuplicate()
- searchVehicle()
- calculateTotal()
- registerVehicle()
- displayRecord()
- displayAll()
- updateServiceRecord()
- updateVehicle()
- calculateBill()
- sortByServiceCost()
- displayHighCostVehicles()

- generateDailySummary()
- saveRecords()
- loadRecords()

Each function performs a specific operation within the system.

The modular design provides several advantages:

- Easier debugging
- Better code organization
- Easier maintenance
- Code reusability
- Simplified testing
- Clear separation of responsibilities among team members

6.6 Complete C Program

```
#include <stdio.h>
```

```
#include <string.h>
```

```
#define MAX 100
```

```
struct Date {  
    int day, month, year;  
};
```

```
struct Vehicle {  
    char vehicleNo[20];  
    char customer[50];  
    char model[30];  
    char service[30];  
  
    struct Date date;
```

```
float parts;

float labour;

float total;
};

void calculate(struct Vehicle *v)
{
    v->total = v->parts + v->labour;
}

void addVehicle(struct Vehicle v[], int *n)
{
    if (*n >= MAX) {
        printf("Maximum records reached!\n");
        return;
    }

    printf("\nEnter Vehicle Number: ");
    scanf("%s", v[*n].vehicleNo);

    printf("Enter Customer Name: ");
    scanf(" %[^\\n]", v[*n].customer);

    printf("Enter Vehicle Model: ");
    scanf(" %[^\\n]", v[*n].model);
```

```
printf("Enter Service Type: ");

scanf("%[^\\n]", v[*n].service);


printf("Enter Date (DD MM YYYY): ");

scanf("%d %d %d",

    &v[*n].date.day,

    &v[*n].date.month,

    &v[*n].date.year);


printf("Enter Parts Cost: ");

scanf("%f", &v[*n].parts);


printf("Enter Labour Cost: ");

scanf("%f", &v[*n].labour);


if (v[*n].parts < 0 || v[*n].labour < 0) {

    printf("Invalid cost!\\n");

    return;

}


calculate(&v[*n]);


printf("Total Cost = Rs. %.2f\\n", v[*n].total);
```

```

    (*n)++;

    printf("Vehicle registered successfully!\n");
}

int search(struct Vehicle v[], int n, char no[])
{
    int i;

    for (i = 0; i < n; i++) {
        if (strcmp(v[i].vehicleNo, no) == 0)
            return i;
    }

    return -1;
}

void display(struct Vehicle *v)
{
    printf("\nVehicle Number : %s", v->vehicleNo);
    printf("\nCustomer Name : %s", v->customer);
    printf("\nVehicle Model : %s", v->model);
    printf("\nService Type : %s", v->service);
    printf("\nDate : %02d-%02d-%d",
        v->date.day,
        v->date.month,

```

```
        v->date.year);

printf("\nParts Cost    : Rs. %.2f", v->parts);

printf("\nLabour Cost   : Rs. %.2f", v->labour);

printf("\nTotal Cost    : Rs. %.2f\n", v->total);
}
```

```
void searchVehicle(struct Vehicle v[], int n)
```

```
{
    char no[20];

    int pos;

    printf("\nEnter Vehicle Number: ");

    scanf("%s", no);

    pos = search(v, n, no);

    if (pos == -1)

        printf("Vehicle not found!\n");

    else

        display(&v[pos]);
}
```

```
void update(struct Vehicle v[], int n)
```

```
{
    char no[20];
```

```
int pos;

printf("\nEnter Vehicle Number: ");

scanf("%s", no);

pos = search(v, n, no);

if (pos == -1) {

    printf("Vehicle not found!\n");

    return;

}

printf("Enter New Service Type: ");

scanf(" %[^\\n]", v[pos].service);

printf("Enter New Parts Cost: ");

scanf("%f", &v[pos].parts);

printf("Enter New Labour Cost: ");

scanf("%f", &v[pos].labour);

calculate(&v[pos]);

printf("Record updated successfully!\n");

}
```

```
void displayAll(struct Vehicle v[], int n)
{
    int i;

    if (n == 0) {
        printf("No records available!\n");
        return;
    }

    for (i = 0; i < n; i++) {
        printf("\n--- Record %d ---", i + 1);
        display(&v[i]);
    }
}
```

```
void sortVehicles(struct Vehicle v[], int n)
{
    int i, j;
    struct Vehicle temp;

    for (i = 0; i < n - 1; i++) {
        for (j = i + 1; j < n; j++) {

            if (v[i].total < v[j].total) {
```

```
        temp = v[i];

        v[i] = v[j];

        v[j] = temp;

    }

}

printf("\nRecords sorted by highest service cost.\n");

displayAll(v, n);

}
```

```
void highCost(struct Vehicle v[], int n)

{

    float amount;

    int i, found = 0;


    printf("\nEnter Cost Limit: ");

    scanf("%f", &amount);


    for (i = 0; i < n; i++) {

        if (v[i].total > amount) {

            display(&v[i]);

            found = 1;

        }

    }

}
```

```
    if (!found)

        printf("No high-cost services found.\n");
}
```

```
void save(struct Vehicle v[], int n)
```

```
{
    FILE *fp;

    fp = fopen("service_records.dat", "wb");

    if (fp == NULL) {
        printf("File error!\n");
        return;
    }
```

```
    fwrite(&n, sizeof(int), 1, fp);
    fwrite(v, sizeof(struct Vehicle), n, fp);
```

```
    fclose(fp);
```

```
    printf("Records saved successfully!\n");
}
```

```
void load(struct Vehicle v[], int *n)
```

```
{  
  
    FILE *fp;  
  
    fp = fopen("service_records.dat", "rb");  
  
    if (fp == NULL) {  
        printf("No previous records found.\n");  
        return;  
    }  
  
    fread(n, sizeof(int), 1, fp);  
    fread(v, sizeof(struct Vehicle), *n, fp);  
  
    fclose(fp);  
  
    printf("%d records loaded.\n", *n);  
}  
  
int main()  
{  
    struct Vehicle v[MAX];  
  
    int n = 0;  
  
    int choice;  
  
    load(v, &n);
```

```
do {

    printf("\n\n=====");

    printf("\n AUTOMOBILE SERVICE CENTRE");

    printf("\n=====");

    printf("\n1. Register Vehicle");

    printf("\n2. Search Vehicle");

    printf("\n3. Update Record");

    printf("\n4. Display All Records");

    printf("\n5. Sort by Service Cost");

    printf("\n6. High-Cost Services");

    printf("\n7. Save Records");

    printf("\n8. Exit");


    printf("\nEnter Choice: ");

    scanf("%d", &choice);


    switch (choice) {

        case 1:

            addVehicle(v, &n);

            break;


        case 2:

            searchVehicle(v, n);
```

```
break;
```

```
case 3:
```

```
    update(v, n);
```

```
    break;
```

```
case 4:
```

```
    displayAll(v, n);
```

```
    break;
```

```
case 5:
```

```
    sortVehicles(v, n);
```

```
    break;
```

```
case 6:
```

```
    highCost(v, n);
```

```
    break;
```

```
case 7:
```

```
    save(v, n);
```

```
    break;
```

```
case 8:
```

```
    save(v, n);
```

```
    printf("Thank you!\n");
```

```

        break;

    default:

        printf("Invalid choice!\n");
    }

} while (choice != 8);

return 0;
}

```

6.7 Explanation of Pointer Usage

A key requirement of the assignment is to demonstrate the practical use of **pointers to structures**. In the proposed system, pointers are used to directly access and modify service records.

The function:

```
void calculateTotal(ServiceRecord *record)
```

receives the memory address of a ServiceRecord structure. The pointer refers directly to the original structure, allowing its members to be accessed and modified without creating a separate copy.

The structure members are accessed using the arrow operator (->):

```

record->partsCost
record->labourCost
record->totalServiceCost

```

The arrow operator is used because record is a pointer to a structure.

The function is called using:

```
calculateTotal(&newRecord);
```

Here, the address-of operator (&) passes the memory address of newRecord to the function.

The pointer-based approach provides several advantages:

- Avoids copying the entire structure.
- Allows direct modification of the original record.
- Reduces unnecessary memory usage.
- Improves efficiency when working with larger structures.
- Demonstrates the practical application of pointers in a real-world program.

The update operation also uses a pointer-based approach through the function:

```
void updateServiceRecord(ServiceRecord *record)
```

The function is called as:

```
updateServiceRecord(&records[index]);
```

As a result, any changes made inside the function are applied directly to the original service record stored in the array.

6.8 Implementation Summary

The implementation combines several important **C programming concepts** to satisfy the functional requirements of the Automobile Service Centre Management System.

The Automobile Service Centre Management System is implemented by combining several fundamental concepts of the C programming language. **Structures** are used to organize vehicle, customer, service, and billing information into individual records. **Nested structures** help represent detailed information such as service dates within the main service record. **Arrays** are used to maintain multiple vehicle service records during program execution.

Pointers are applied for accessing record addresses and modifying service information directly. **Functions** divide the program into separate modules, making operations such as registration, searching, updating, sorting, and reporting easier to manage. **Linear search** is used to locate a vehicle by checking its registration number against the stored records. **Insertion sort** is implemented to arrange service records according to their total service cost in descending order.

The system also uses **file handling** to store records permanently and retrieve them when the program is restarted. **Input validation** ensures that incorrect dates, negative costs, duplicate vehicle numbers, and other invalid values are not accepted. **Error handling** is incorporated to manage situations such as full record capacity, missing vehicle records, invalid input, and file-related problems.

Overall, the implementation demonstrates the practical application of important C programming concepts in developing a functional automobile service management system. The combination of data structures, pointers, functions, searching, sorting, file operations, validation, and error handling makes the system organized, reliable, and efficient.

7. Error Handling Mechanism

The Automobile Service Centre Management System uses an error-handling mechanism to prevent invalid inputs and ensure smooth program execution. The system validates user inputs

before performing operations and displays suitable error messages when incorrect data is entered.

- Invalid menu choice: Displays an error message and asks the user to enter a valid option.
- Duplicate vehicle number: Prevents registration of the same vehicle more than once.
- Negative service cost: Rejects negative values for parts and labour costs.
- Invalid service date: Checks whether the entered date is valid before storing the service record.
- Vehicle not found: Displays an appropriate message when a searched vehicle number does not exist.
- Invalid input type: Handles non-numeric input where a numeric value is expected.
- File handling errors: Checks whether service records can be successfully saved and loaded.
- Safe program execution: Prevents unexpected termination by validating inputs before processing them.

Overall, the error-handling mechanism improves reliability, prevents incorrect records, and provides clear feedback to the user.

9. Test Cases and Result

Test Case 1: Program Startup And Menu

OUTPUT:

```
====
                        AUTOMOBILE SERVICE CENTRE MANAGEMENT SYSTEM (C-BASED)
=====
====
Success: Loaded 4 previous service record(s) from 'service_records.dat'.
=====
====
                        MAIN MENU
=====
====
1. Register a Vehicle for Service
2. Search Vehicle by Vehicle Number
3. Update Service Details
4. Calculate Total Bill (Generate Invoice)
5. Sort Vehicles based on Service Cost (QuickSort)
6. Display Vehicles Exceeding Specified Cost Threshold
7. Generate Daily Service Summary Report
8. Store Service Records to File
9. Reload Previous Records from File
0. Save & Exit Application
=====
====
Enter your choice (0-9): █
```

Test Case 2: Successful Registration of Normal Vehicle Service

OUTPUT:

```
=====
====
1. Register a Vehicle for Service
2. Search Vehicle by Vehicle Number
3. Update Service Details
4. Calculate Total Bill (Generate Invoice)
5. Sort Vehicles based on Service Cost (QuickSort)
6. Display Vehicles Exceeding Specified Cost Threshold
7. Generate Daily Service Summary Report
8. Store Service Records to File
9. Reload Previous Records from File
0. Save & Exit Application
=====
====
Enter your choice (0-9): 1

--- REGISTER NEW VEHICLE ---
Enter Vehicle Number (e.g. KA01AB1234): AP40LR2700
Enter Customer Name: Jaswanth
Enter Vehicle Model (e.g. Honda City): Suzuki Brezza
Enter Service Type (e.g. Full Service, Oil Change): Oil change
Enter Service Date (DD-MM-YYYY): 02-09-2026
Enter Parts Cost (INR): 2000
Enter Labour Cost (INR): 500
Success: Vehicle 'AP40LR2700' registered successfully! Total Cost: INR 2500.00
=====
```

Test Case 3: Registration of High-Cost Engine Repair Service

OUTPUT:

```
=====
====
1. Register a Vehicle for Service
2. Search Vehicle by Vehicle Number
3. Update Service Details
4. Calculate Total Bill (Generate Invoice)
5. Sort Vehicles based on Service Cost (QuickSort)
6. Display Vehicles Exceeding Specified Cost Threshold
7. Generate Daily Service Summary Report
8. Store Service Records to File
9. Reload Previous Records from File
0. Save & Exit Application
=====
====
Enter your choice (0-9): 1

--- REGISTER NEW VEHICLE ---
Enter Vehicle Number (e.g. KA01AB1234): AP40LR2700
Enter Customer Name: Jaswanth
Enter Vehicle Model (e.g. Honda City): Suzuki Brezza
Enter Service Type (e.g. Full Service, Oil Change): Oil change
Enter Service Date (DD-MM-YYYY): 02-09-2026
Enter Parts Cost (INR): 2000
Enter Labour Cost (INR): 500
Success: Vehicle 'AP40LR2700' registered successfully! Total Cost: INR 2500.00
=====
```

Test Case 4: Searching and Displaying an Existing Vehicle Record

OUTPUT:

```

=====
Enter your choice (0-9): 2

--- SEARCH VEHICLE RECORD ---
Enter Vehicle Number to Search: AP40LR2700

Vehicle Record Found:
+-----+-----+-----+-----+-----+
| S# | Veh Number | Customer Name | Vehicle Model | Service Type |
| Date | Parts(INR) | Labour(INR) | Total(INR) | |
+-----+-----+-----+-----+-----+
| 5 | AP40LR2700 | Jaswanth | Suzuki Brezza | Oil change |
| 02-09-2026 | 2000.00 | 500.00 | 2500.00 | |
+-----+-----+-----+-----+-----+

```

Test Case 5: Updating Service Details Using Pointer to Structure

OUTPUT:

```

=====
Enter your choice (0-9): 3

--- UPDATE SERVICE DETAILS ---
Enter Vehicle Number to Update: AP40LR2700

Current Details for AP40LR2700:
+-----+-----+-----+-----+-----+
| S# | Veh Number | Customer Name | Vehicle Model | Service Type |
| Date | Parts(INR) | Labour(INR) | Total(INR) | |
+-----+-----+-----+-----+-----+
| 1 | AP40LR2700 | Jaswanth | Suzuki Brezza | Oil change |
| 02-09-2026 | 2000.00 | 500.00 | 2500.00 | |
+-----+-----+-----+-----+-----+

Enter Updated Details:
Enter New Vehicle Model: Suzuki Brezza
Enter New Service Type: Full Service
Enter New Service Date (DD-MM-YYYY): 02-09-2026
Enter New Parts Cost (INR): 10000
Enter New Labour Cost (INR): 2000
Success: Vehicle service details updated successfully!

```

Test Case 6: Vehicles Sorted from Highest to Lowest Service Cost

OUTPUT:

```

=====
Enter your choice (0-9): 4

--- CALCULATE TOTAL BILL ---
Enter Vehicle Number for Invoice: AP40LR2700

=====
AUTOMOBILE SERVICE INVOICE
=====
Vehicle Number : AP40LR2700
Customer Name : Jaswanth
Model : Suzuki Brezza
Service Type : Full Service
Service Date : 02-09-2026
-----
Parts Cost : INR 10000.00
Labour Cost : INR 2000.00
-----
TOTAL BILL : INR 12000.00
=====

```

Test Case 7: Displaying Vehicles Above the Specified Service Cost

OUTPUT:

```
====
Enter your choice (0-9): 4

--- CALCULATE TOTAL BILL ---
Enter Vehicle Number for Invoice: AP40LR2700

=====
                AUTOMOBILE SERVICE INVOICE
=====
Vehicle Number : AP40LR2700
Customer Name  : Jaswanth
Model          : Suzuki Brezza
Service Type   : Full Service
Service Date   : 02-09-2026
-----
Parts Cost    : INR   10000.00
Labour Cost   : INR    2000.00
-----
TOTAL BILL    : INR   12000.00
=====
```

Test Case 8: Daily Service Summary and Total Revenue Calculation

OUTPUT:

```
Enter your choice (0-9): 5
--- SORT VEHICLES BY SERVICE COST ---
Success: Sorted 5 vehicle record(s) by total service cost (Descending Order).

+-----+
| S# | Veh Number | Customer Name | Vehicle Model | Service Type | Date | Parts(INR) | Labour(INR) |
+-----+-----+-----+-----+-----+-----+-----+
| 1 | KA03EF0012 | Vikram Singh | Toyota Fortuner | Engine Overhaul | 15-08-2026 | 35000.00 | 15000.00 |
| 2 | AP40LR2700 | Jaswanth | Suzuki Brezza | Full Service | 02-09-2026 | 10000.00 | 2000.00 |
| 3 | KA01AB1234 | Ramesh Kumar | Honda City | Full Service | 15-08-2026 | 4500.50 | 1500.00 |
| 4 | KA06G01111 | Amit Patel | Maruti Swift | Brake Repair | 15-09-2026 | 2000.00 | 500.00 |
| 5 | KA02CD5678 | Sita Sharma | Hyundai i20 | Oil Change | 20-08-2026 | 1200.00 | 800.00 |
+-----+-----+-----+-----+-----+-----+-----+

```

Test Case 9: Loading Previous Service Records When the Program Restarts

OUTPUT:

```
Enter your choice (0-9): 9
Success: Loaded 5 previous service record(s) from 'service_records.dat'.
```

```
=====
Enter your choice (0-9): 9
Success: Loaded 5 previous service record(s) from 'service_records.dat'.
```

10. Analysis and Discussion

The Automobile Service Centre Management System was developed to provide a structured computerized solution for managing vehicle service records and to demonstrate the practical application of fundamental C programming concepts. The system stores important details such as vehicle number, customer name, vehicle model, service type, service date, parts cost, labour

cost, and total service cost. Structures and arrays of structures are used to organize and manage multiple service records efficiently. Pointers to structures are used for operations such as calculating service costs and updating existing records, allowing the original records to be modified directly without unnecessary copying. The total service cost is calculated using the formula $\text{Total Service Cost} = \text{Parts Cost} + \text{Labour Cost}$.

The system uses Linear Search to locate vehicle records based on their vehicle numbers. This approach is simple and suitable for the limited number of records handled by the application. Insertion Sort is used to arrange vehicles according to their service costs in descending order, making it easier to identify high-value services. Since the system is designed for a relatively small number of records, these algorithms provide acceptable performance while keeping the implementation simple and understandable.

File handling is another important feature of the system. Service records are stored in a binary file so that information is not lost when the application is closed. The functions `fwrite()` and `fread()` are used to save and retrieve records, and the record count is maintained to ensure that previously stored records can be loaded correctly when the program starts.

Input validation and error handling improve the reliability and safety of the application. The system rejects duplicate vehicle numbers, negative service costs, invalid service dates, and invalid menu inputs. It also checks the maximum record capacity, verifies file operations, and checks search results before accessing records. These mechanisms prevent incorrect data entry, unsafe memory access, and unexpected program termination.

Testing demonstrated that the system performs its major operations correctly under both normal and invalid input conditions. Vehicle registration and service-cost calculation worked successfully, while duplicate records, negative costs, invalid dates, and invalid menu inputs were handled safely. Searching, updating, sorting, high-cost filtering, daily service summaries, file saving, and file loading also produced the expected results.

The major strength of the system is that it combines several important C programming concepts, including structures, arrays, pointers, functions, searching, sorting, file handling, input validation, and error handling, within one practical application. The menu-driven and modular design also improves readability, usability, maintainability, and testing. However, the fixed-size array limits the number of records, and the current system is mainly intended for a single user without authentication or access control. Future improvements could include GST and discount calculation, mechanic assignment, spare-parts inventory management, invoice generation, service reminders, online booking, database integration, and user authentication.

Overall, the Automobile Service Centre Management System provides a good balance between simplicity, functionality, and performance for an academic project. It successfully demonstrates how a manual vehicle service-record process can be converted into a computerized system using C programming while maintaining proper validation, error handling, runtime safety, and resource management.

11.Conclusion

The Automobile Service Centre Management System was successfully developed using C programming to provide an efficient and organized method for managing vehicle service records. The system replaces traditional manual record keeping with a computerized solution that supports vehicle registration, service-cost calculation, searching, updating, sorting, high-cost service identification, daily revenue calculation, and permanent record storage.

The project effectively demonstrates important C programming concepts such as structures, arrays, pointers, functions, searching, sorting, file handling, input validation, and error handling. The menu-driven interface makes the system simple to operate, while validation mechanisms help prevent incorrect data, duplicate records, invalid dates, and other input errors. File handling also ensures that service records can be saved and retrieved when required.

Overall, the system provides a simple and reliable solution for the basic requirements of an automobile service centre. Although the current version is suitable for small-scale use, future enhancements such as database integration, user authentication, spare-parts management, invoice generation, service reminders, and online booking can make the system more scalable and suitable for real-world applications

References:

1. Kernighan, B. W., & Ritchie, D. M. (1988). The C Programming Language (2nd ed.). Prentice Hall.
2. Balagurusamy, E. (2019). Programming in ANSI C (8th ed.). McGraw Hill Education.
3. Kanetkar, Y. (2016). Let Us C (15th ed.). BPB Publications.
4. Schildt, H. (2017). C: The Complete Reference (4th ed.). McGraw Hill Education.
5. ISO/IEC 9899:2018. Programming Languages — C. International Organization for Standardization.
6. GNU Project. GCC: The GNU Compiler Collection – C Programming Documentation. GNU Project.
7. TutorialsPoint. C Programming Language – Structures, Pointers, Functions, Arrays and File Handling. TutorialsPoint.
8. GeeksforGeeks. C Programming – Structures, Pointers, File Handling and Sorting. GeeksforGeeks.

